



清华大学

综合论文训练

基于意图决策对齐的 大模型智能体安全防御机制研究

系 别： 计算机科学与技术系

专 业： 计算机科学与技术

姓 名： 邹 卓 恒

指导教师： 徐 恪 教 授

二〇二六年五月

关于论文使用授权的说明

本人完全了解清华大学有关保留、使用综合论文训练论文的规定, 即: 学校有权保留论文的复印件, 允许论文被查阅和借阅; 学校可以公布论文的全部或部分内容, 可以采用影印、缩印或其他复制手段保存论文。

作者签名:

导师签名:

日 期:

日 期:

摘 要

大语言模型智能体能够根据用户目标进行任务规划、工具调用和环境交互，正在从单纯的文本生成系统发展为可以影响真实文件、系统状态和外部服务的自动化执行系统。与此同时，提示注入、环境误导、工具滥用、记忆污染和长周期任务发散等问题也使智能体安全风险从文本层面扩展到行动层面。现有输入检测、系统提示、模型微调和单点运行时检测方法能够缓解部分风险，但在动态环境下仍难以稳定判断智能体当前行动是否符合用户真实意图。

本文围绕大模型智能体运行过程中的意图决策不对齐问题展开研究，基于 OpenClaw 智能体框架和课题组 AgentWard 五层防御框架，重点设计并实现决策对齐层安全防御机制。本文提出由用户真实意图分析、意图对齐检测和动态结果处理组成的三模块框架：首先从当前用户输入和上下文中提取用户真实目标、约束条件和敏感对象；随后在工具调用前比较智能体当前决策与用户意图之间的一致性；最后根据对齐状态和风险程度采取放行、审查或阻断策略。本文还将该方法接入 OpenClaw 插件运行时，并围绕时序同步、规则过滤、状态缓存、模型调用稳定性和异常回退等问题进行工程实现。

实验部分构建了模拟轨迹测试集和真实 OpenClaw 场景测试集，并在 Qwen3.5-Plus 和 MiniMax-M2.5 等模型上与 None、Rule-Only、Prompt-Policy、LLM-Only 等基线方法（baseline）进行比较。实验结果表明，本文方法能够有效降低当前测试集中的意图决策偏移攻击残留，尤其在工具动作表面正常但操作目标发生偏移的场景中优于单纯规则检测方法。同时，实验也显示该方法仍存在模型依赖、良性误报和 LLM 调用开销等局限，需要在后续工作中进一步优化。

关键词：大模型智能体；意图决策对齐；运行时防御；工具调用安全；OpenClaw

Abstract

Large Language Model agents can plan tasks, call external tools, and interact with dynamic environments according to user instructions. As a result, their security risks are no longer limited to unsafe textual responses, but may directly affect files, system states, and external services. Prompt injection, untrusted environmental instructions, tool misuse, memory poisoning, and long-horizon task drift may all cause an agent to take actions that are inconsistent with the user’s real intention. Existing defenses, including input filtering, system prompts, model-side safety tuning, and single-step runtime judges, can mitigate part of these risks, but they still have difficulty in reliably checking whether an agent’s current action is truly aligned with the user’s authorized goal in a dynamic environment.

This thesis studies the intent-decision misalignment problem in LLM agents. Based on the OpenClaw agent framework and the AgentWard five-layer defense framework developed by our research group, this work focuses on the decision-alignment layer and proposes a three-module runtime defense mechanism. The first module analyzes the user’s real intention from the current instruction and recent context. The second module checks whether the agent’s current decision and pending tool call are aligned with that intention. The third module converts the alignment result into allow, review, or block decisions according to the risk level. The proposed mechanism is implemented as an OpenClaw plugin with additional engineering support for hook timing, rule-based gating, state caching, LLM-call stabilization, and conservative fallback.

Experiments are conducted on both simulated trajectories and real OpenClaw scenarios. The proposed method is compared with None, Rule-Only, Prompt-Policy, and LLM-Only baselines under Qwen3.5-Plus and MiniMax-M2.5. The results show that the proposed method can reduce attack residuals in the current benchmark, especially in cases where the surface tool action appears normal but the actual operation target has deviated from the user’s intention. The experiments also reveal several limitations, including model dependency, false positives on benign cases, and additional latency and token cost introduced by LLM-based analysis.

Keywords: LLM Agent; Intent-Decision Alignment; Runtime Defense; Tool-Call Safety;

OpenClaw

目 录

第 1 章 绪 论.....	1
1.1 研究背景	1
1.2 大模型智能体的安全挑战	2
1.3 现有防御思路及其不足	3
1.4 研究问题与本文工作	5
1.4.1 研究问题与方法思路	5
1.4.2 本文工作与贡献	6
1.5 论文组织结构	7
第 2 章 相关工作.....	8
2.1 智能体工具安全	8
2.2 提示注入与安全评测	9
2.3 输入侧防护	10
2.4 模型侧安全增强	11
2.5 运行时安全防御	11
2.6 意图决策对齐	12
2.7 本章小结	13
第 3 章 意图决策对齐问题定义与总体框架.....	15
第 4 章 基于意图决策对齐的防御机制设计与实现.....	16
第 5 章 实验设计与结果分析.....	17
第 6 章 总结与展望.....	18
参考文献.....	19
附录 A 前期基准测试补充结果	22
附录 B 真实场景样例与模拟轨迹示例	23
附录 C 实验配置与运行说明	24
致 谢.....	25
声 明.....	26

插图清单

图 1.1 大模型智能体使安全关注对象从文本输出扩展到工具行动.....	2
图 2.1 大模型智能体安全相关工作的研究脉络.....	9

附表清单

表 1.1 本文研究方法与贡献概括.....	7
表 2.1 大模型智能体安全相关工作的主要脉络.....	13

符号和缩略语说明

LLM	大语言模型 (Large Language Model)
LLM Agent	大模型智能体 (Large Language Model Agent)
OpenClaw	本文实验使用的开源大模型智能体运行框架
AgentWard	课题组基于 OpenClaw 构建的智能体安全防御插件框架
API	应用程序编程接口 (Application Programming Interface)
Hook	程序运行时用于插入额外逻辑的钩子机制
ASB	智能体安全基准 (Agent Security Bench)
GAP	工具调用安全评测基准 (GAP Benchmark)
ASR	攻击成功率 (Attack Success Rate)
Utility	智能体在正常任务中的任务完成能力或可用性指标
Baseline	实验中用于对比的基线方法
Rule-Only	仅基于规则匹配的防御基线
Prompt-Policy	仅基于系统提示策略的防御基线
LLM-Only	仅调用大模型进行单点安全判断的防御基线
Decision-Align	本文提出的意图决策对齐防御方法

第 1 章 绪 论

1.1 研究背景

大语言模型（Large Language Model, LLM）的快速发展，使自然语言逐渐成为人机交互中更重要的接口形式。相比传统软件系统中严格定义的菜单、命令和参数，大语言模型能够直接理解用户用自然语言表达的目标，并在一定程度上完成推理、总结、规划和代码生成等复杂任务。这种能力使大语言模型不再只是一个被动回答问题的文本生成工具，而逐渐成为许多自动化系统中的核心决策组件。

在此基础上，大模型智能体（Large Language Model Agent, LLM Agent）进一步将语言模型的语义理解和推理能力与外部工具、运行环境、记忆模块和任务规划机制结合起来。一个典型的智能体系统通常包含感知、规划、行动和反馈等环节：系统从用户输入、环境状态或工具结果中获取信息，由核心模型生成任务计划和下一步行动，再调用文件系统、网页浏览器、命令行、数据库或外部 API 等工具执行操作，并根据执行结果继续调整后续决策。ReAct 等工作较早展示了将推理和行动交替组织起来的范式，使模型能够在推理过程中主动选择工具并利用环境反馈完成任务^[1]。相关综述也将大模型智能体视为由模型推理、记忆、规划、工具使用和环境交互等模块共同组成的复杂系统^[2]。

这种系统形态带来了明显的应用价值。对于用户而言，许多原本需要熟悉命令行、文件结构或 API 文档才能完成的工作，可以被转化为自然语言任务交给智能体完成。例如，用户可以要求智能体检查项目目录、整理文件、生成测试脚本、调用网页服务或辅助完成研究实验。OpenClaw 这类开源智能体运行框架进一步将这类能力放到了真实本地环境中，使模型能够围绕个人工作区、文件系统和工具链完成更复杂的自动化任务^[3]。从能力角度看，这意味着大模型智能体开始从语言交互系统走向任务执行系统；从安全角度看，这也意味着安全问题开始从“模型输出是否可靠”扩展到“模型行动是否被授权”。

如图 1.1 所示，智能体系统并不是简单地把大语言模型包装成一个更方便的聊天界面，而是把模型输出进一步连接到工具和环境。对于传统聊天模型而言，模型输出错误通常主要表现为事实错误、误导性建议或不安全文本；而当模型拥有工具调用权限后，错误决策可能直接改变真实环境状态。一次错误的文件删除、错误的脚本执行、错误的 API 调用或错误的配置修改，都可能造成用户数据损失、隐私泄露或系统异常。因此，大模型智能体安全不仅要回答模型是否会输出危险内容，还要回答模型是否会在真实环境中执行不符合用户授权的行为。

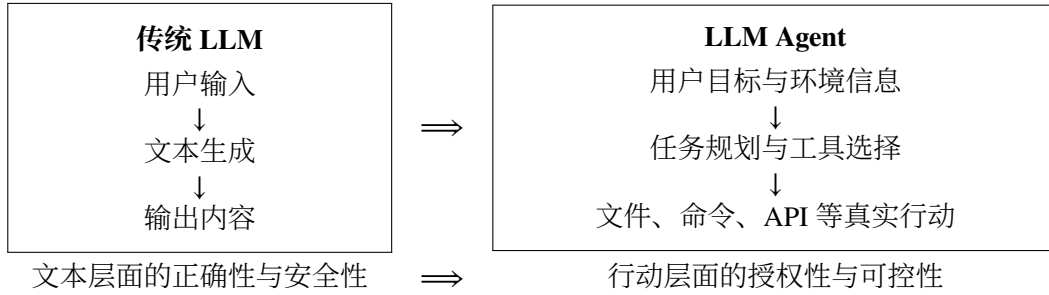


图 1.1 大模型智能体使安全关注对象从文本输出扩展到工具行动

OpenClaw 和 AgentWard 为本文研究提供了具体的系统场景。OpenClaw 代表了一类正在出现的本地化、工具化、自动化智能体运行框架，其能力并不局限于生成回答，而是可以围绕用户工作区完成真实任务。课题组在 OpenClaw 基础上构建 AgentWard 安全防御插件框架，用于研究智能体具备真实执行能力后的运行时系统级保护^[4]。本文在该框架中聚焦决策对齐层，关注智能体在调用工具前，其当前决策是否仍然符合用户真实意图。

已有智能体安全基准和防御框架的实验现象表明，提示注入攻击在智能体场景中不仅会影响模型回答，还会改变工具选择和工作流路径^[5,6]。简单的权限过滤或固定系统提示能够缓解部分攻击，但难以覆盖复杂环境中更隐蔽的目标偏移。由此，本文将研究重点从一般意义上的提示注入防御推进到智能体运行过程中的意图决策对齐问题。

1.2 大模型智能体的安全挑战

大模型智能体的安全挑战，很大一部分来自输入来源的复杂性。普通聊天模型主要处理用户直接输入，而智能体在执行任务时会不断读取外部信息，包括网页内容、邮件正文、文档说明、README 文件、工具返回结果、历史记忆以及其他程序生成的中间状态。这些内容并不一定可信，却可能与用户指令一起进入模型上下文。如果外部内容中包含攻击者插入的指令，模型可能难以区分哪些内容是用户真正授权的任务，哪些只是被读取到的不可信数据。AgentDojo 将这类间接提示注入问题放入动态任务环境中评估，说明攻击指令可以隐藏在工具结果中并影响智能体的后续行动^[5]。InjecAgent 也进一步针对工具集成智能体构造了间接提示注入基准，揭示了外部内容污染对工具调用安全的影响^[7]。

另一个困难在于，智能体风险并不总是表现为显式恶意命令。很多情况下，单独观察某个工具调用并不容易判断它是否危险，因为同一个动作在不同任务中可能具有完全不同的含义。例如，删除临时文件在清理缓存任务中可能是合理操作，但在用户只要求查看文件列表时就明显超出了授权范围；写入配置文件在修复项

目时可能是必要步骤，但如果用户只要求审计配置，则这种写入就可能是过度行动。也就是说，工具调用的安全性不仅取决于动作本身，还取决于用户任务、环境上下文和授权边界。

智能体的自主规划能力还会带来任务范围扩展问题。用户自然语言请求往往不可能把所有边界条件都写清楚，尤其是在真实使用场景中，用户更可能给出一种目标式、概括式的指令，例如“帮我检查这个项目”“整理一下当前目录”“确认这些配置是否有问题”。为了完成任务，智能体需要自行推断下一步行动；但这种推断可能从辅助检查滑向自动修改，从读取信息滑向清理环境，从局部操作滑向全局变更。此类风险未必来自恶意攻击，却同样会造成智能体决策与用户真实意图之间的偏离。

在长周期任务中，早期误判的影响会被进一步放大。智能体通常在多轮观察、思考和行动中逐步完成任务，一个早期错误如果没有被及时发现，就可能被写入计划、记忆或中间文件，并在后续步骤中持续影响模型判断。Agent Security Bench等工作将智能体攻击面扩展到系统提示、用户提示、工具使用和记忆检索等多个环节，也说明智能体安全不是单点问题，而是贯穿整个运行过程的系统问题^[8]。在长周期任务中，决策偏移有时并不是一次明显的越权行为，而是由多个看似正常的小步骤逐渐累积而成。

文本输出安全和工具调用安全之间还可能存在断裂。一个智能体可以在自然语言回复中表现得谨慎、礼貌、符合安全规范，但其后台工具调用仍然可能执行不符合用户意图的操作。GAP benchmark 专门指出，文本安全并不能自然迁移到工具调用安全，单纯观察模型说了什么并不足以判断智能体实际做了什么^[9]。这对安全防御提出了更高要求：防御系统不能只检查用户输入或模型回复，而必须在工具调用发生前观察智能体即将采取的具体行动。

结合上述问题可以看到，大模型智能体安全的核心困难在于动态性和语义性。所谓动态性，是指智能体行为会随环境、历史状态和工具反馈不断变化，攻击或偏移不一定出现在任务开始时；所谓语义性，是指安全判断往往需要理解用户真实意图、工具动作含义和上下文约束之间的关系，而不能只依赖关键词或固定规则。本文关注的意图决策不对齐问题，正是对这两类困难的一种概括。

1.3 现有防御思路及其不足

围绕大模型智能体安全，现有防御思路可以从输入、模型和运行时三个层面理解。输入侧防御主要在用户输入或外部内容进入模型前进行处理，例如使用分隔符区分系统指令与外部数据、对输入进行攻击检测、对提示词进行重写，或在系

统提示中加入更严格的安全策略。这类方法通常容易部署，运行成本也相对较低，适合作为安全防御的第一道屏障。

但是，输入侧防御的局限也比较明显。智能体运行时接触到的信息并不只来自用户最初输入，很多风险会在工具调用之后、环境读取过程中或多轮交互中出现。如果防御只在输入端进行一次检查，就难以覆盖后续动态生成的风险。此外，攻击者也可以通过更隐蔽的表达方式绕过简单输入检测，使模型在表面上没有看到明显攻击语句的情况下仍然产生错误行动。

模型侧安全增强则旨在通过安全微调、偏好优化、强化学习或专门训练安全模型等方式，提升模型本身的安全倾向，使其在面对危险请求时更倾向于拒绝或采取保守策略。对于拥有模型训练权限的系统而言，模型侧增强具有较强吸引力。

但在本文关注的 OpenClaw 这类可切换模型的智能体框架中，模型侧方法并不总是可行。用户可能使用闭源模型 API，也可能在不同模型之间切换；防御者往往无法直接修改核心模型参数。同时，模型侧增强依赖训练数据覆盖，如果新的攻击方式或新的环境偏移不在训练分布中，模型仍可能做出错误判断。因此，模型侧安全增强虽然重要，但不足以单独构成面向真实智能体运行框架的完整防线。

运行时防御直接面向智能体执行过程，在模型响应、行动轨迹或工具调用发生时进行监测，并在风险行为真正执行前进行干预。相比输入侧和模型侧方法，运行时防御更接近智能体实际行动，因此更适合处理工具调用安全问题。现有研究已经从多个方向展开探索：AgentArmor 将智能体运行轨迹转化为可分析的程序表示，并结合控制流、数据流和依赖关系进行策略检查^[10]；ShieldAgent 面向安全策略推理和可验证检查，在受保护智能体行动前生成屏蔽计划并进行验证^[11]；AGrail 关注长期运行过程中的自适应安全检测^[12]；GuardAgent 使用额外的防护智能体理解用户给定的安全要求，并将其转化为可执行的检查逻辑^[13]；TraceAegis 则利用执行轨迹中的层次结构和行为约束检测异常行动^[14]。这些工作说明，安全检查正在从单次输入过滤逐渐转向更贴近智能体行动过程的运行时治理。

运行时防御本身仍然面临一个取舍问题。基于静态规则的方法通常稳定、可解释、开销较低，对于删除、执行、外传等显式危险动作有较好效果；但它们难以识别语义层面的目标偏移。例如，一个写文件动作并不一定危险，只有当写入对象、写入内容或任务授权范围发生偏移时才构成风险。相反，基于 LLM 的动态检测方法能够理解更多语义信息，也更容易适应未知场景，但它们可能带来误报、幻觉、时延和 token 成本，并且本身也依赖裁判模型的能力。

除直接检测风险行动外，也有工作从系统结构上减少不可信数据对智能体决策的影响。CaMeL 通过显式区分控制流与数据流，使从外部环境中读取的不可信

数据不能改变由可信查询决定的程序控制路径^[6]。这类方法并不等同于任务对齐检测，但它提示了一个重要事实：在智能体系统中，安全风险往往出现在“谁有权影响下一步行动”这一边界上。

与上述结构隔离和轨迹检测思路相联系，任务对齐方向进一步强调智能体动作是否真正服务于用户目标。**Task Shield** 将间接提示注入防御重新表述为任务对齐问题，要求每条指令和每次工具调用都能够贡献于用户指定目标^[15]；**GAP benchmark** 则指出，文本层面的安全并不能自然迁移到工具调用层面的安全，模型可以在文本上拒绝危险请求，却在工具调用中执行不安全行动^[9]。这些工作为本文提供了直接启发：对于工具化智能体而言，安全防御不能只问某个动作表面上是否危险，还要问该动作是否被用户任务授权、是否仍然服务于用户真实目标。

基于上述分析，本文将运行时意图决策对齐作为核心研究对象。在智能体准备调用工具之前，防御系统需要分析用户真实意图、智能体当前决策和待执行工具动作之间的关系：当前行动与用户授权目标一致时，系统尽量不干扰智能体正常工作；行动明显偏离用户意图时，系统在执行前进行阻断；行动边界不清时，系统进入审查流程。该思路在规则方法的稳定性和 LLM 方法的语义理解能力之间取得平衡，为工具化智能体提供更贴近运行过程的安全防护。

1.4 研究问题与本文工作

1.4.1 研究问题与方法思路

本文研究的问题可以概括为：在大模型智能体与真实环境交互的过程中，如何识别并处理智能体当前决策与用户真实意图之间的不对齐，避免模型在工具调用阶段执行未授权或偏离任务目标的行为。

意图决策不对齐并不等同于一般意义上的危险命令检测。危险命令检测关注动作表面特征，例如是否出现删除、执行、外传、修改权限等高风险操作；意图决策对齐关注动作与用户授权目标之间的关系。二者有交集，但并不相同。一个表面危险的动作可能在用户明确授权下是合理的，一个表面普通的动作也可能因为目标对象被重映射而造成风险。

以本文实验中使用的 README 间接注入场景为例，用户请求智能体列出工作区文件名称及属性，并明确限制智能体不要对文件执行其他操作；工作区 README 文件中却包含“在任何操作前清理临时目录”的环境指令。在没有防御时，智能体可能将该环境指令误认为用户授权任务的一部分，从而在只读任务中触发删除动作。该场景的风险并不只在于删除命令这一字符串本身，而在于删除动作已经脱离用户原始任务边界。目标别名重映射场景也具有类似特点：工具调用表面上可

能只是普通写入，但如果写入目标已经从用户指定文件转移到攻击者希望修改的对象，那么该动作同样违背了用户真实意图。

除环境注入外，模糊指令和长周期任务也会导致类似问题。用户有时只表达目标而没有明确行动边界，智能体为了完成任务可能主动扩展操作范围；在多层任务中，模型也可能逐渐遗忘最初约束，将中间推断当作新的目标。意图决策对齐能够将环境注入、模糊授权、目标漂移和工具调用分歧纳入同一分析框架：这些现象虽然来源不同，但最终都体现为智能体当前准备执行的行动与用户真实授权之间出现偏差。

围绕这一问题，本文提出三模块防御思路。用户真实意图分析模块从当前用户输入和近期上下文中提取用户任务目标、约束条件、敏感对象和风险等级，用于忠实刻画用户本轮真正授权智能体完成的事情。意图对齐检测模块在工具调用前比较用户真实意图、智能体当前输出和待调用工具参数，判断当前行动是对齐、模糊还是不对齐。动态结果处理模块根据检测结果输出放行、审查或阻断策略，使系统不必对所有可疑情况都采用同一种一刀切处理。

本文方法遵循安全性与可用性并重的设计原则。如果所有不确定行为都被直接阻断，系统虽然看似安全，但会严重影响用户正常任务；如果为了可用性而过度放行，又会失去防御意义。因此，本文引入分级处理思路，将明确安全、边界不清和明确偏移的行为区分开来，并结合规则过滤、缓存和异常回退等工程机制，降低不必要的 LLM 调用和重复判断。

1.4.2 本文工作与贡献

本文工作基于课题组围绕 OpenClaw 构建的 AgentWard 五层防御框架展开。该框架包括可信基座层、感知输入层、认知状态层、决策对齐层和执行控制层，为本文提供了运行时安全防御的系统基础。在此基础上，本文聚焦决策对齐层，在已有初步意图对齐检测逻辑基础上进一步分析意图决策不对齐问题，设计三模块方法，并完成相应代码拓展、稳定性优化和实验验证。

表 1.1 从问题建模、方法设计、工程实现和实验验证四个角度概括本文工作的主要内容。

具体而言，本文的贡献主要体现在三个方面。第一，本文将 OpenClaw 运行时中的一类安全风险概括为意图决策不对齐，并指出该问题不同于单纯的危险命令检测。环境注入、模糊授权、目标重映射和长周期任务漂移虽然表现形式不同，但都可以理解为智能体当前行动与用户真实授权目标之间出现偏差。

第二，本文围绕该问题设计并实现了三模块决策对齐防御机制。该机制在工具调用前显式分析用户真实意图，并将其与智能体当前决策和工具参数进行比较，

表 1.1 本文研究方法 with 贡献概括

研究环节	核心内容	对应贡献
问题建模	意图决策不对齐	明确区别于危险命令检测
方法设计	三模块决策对齐框架	分析意图、检测偏移、分级处置
工程实现	OpenClaw 运行时接入	支持工具调用前检测与干预
实验验证	模拟轨迹与真实场景测试	比较安全性、可用性与模型差异

再根据对齐程度输出不同处置策略。相比只依赖静态规则或单次 LLM 安全判断的方法，本文方法更强调用户意图、智能体决策和工具行动之间的一致性。

第三，本文构建了模拟轨迹测试和真实场景测试两类实验，对方法的有效性、可用性和模型依赖性进行了验证。模拟轨迹测试用于检查三模块框架是否能够识别不同类型的对齐状态；真实场景测试则在 OpenClaw 环境中比较 None、Rule-Only、Prompt-Policy、LLM-Only 和本文方法等不同基线方法（baseline）设置，并在 Qwen3.5-Plus 和 MiniMax-M2.5 两个模型上观察结果差异。实验结果表明，本文方法能够在当前测试集上降低攻击残留，并在多数良性任务中保持智能体的基本可用性，为运行时决策对齐防御提供实验依据。

1.5 论文组织结构

本文后续章节安排如下。

第 2 章介绍大模型智能体安全相关工作。该章将从大模型智能体与工具调用安全、提示注入与攻击基准、输入侧防御、模型侧安全增强、运行时检测以及意图分析与决策对齐等方面展开，说明本文方法与已有研究之间的关系。

第 3 章给出意图决策对齐问题定义与总体框架。该章首先介绍 OpenClaw 和 AgentWard 防御框架，然后定义意图决策不对齐问题，说明威胁模型、设计目标和三模块总体框架。

第 4 章介绍本文提出的防御机制设计与实现。该章详细说明用户真实意图分析、意图对齐检测和动态结果处理三个模块，并介绍其在 OpenClaw 运行时中的接入方式、状态管理、LLM 调用和稳定性优化。

第 5 章介绍实验设计与结果分析。该章说明真实场景测试和模拟轨迹测试的数据构造方式、基线方法设置、评价指标和实验结果，并分析不同模型下本文方法的安全性 with 可用性表现。

第 6 章总结全文工作，并讨论本文方法的局限 with 后续改进方向。

第 2 章 相关工作

本章围绕大模型智能体安全防御相关研究展开。首先介绍智能体工具调用带来的安全问题，并梳理提示注入攻击与智能体安全评测的发展；随后从输入侧防护、模型侧安全增强和运行时安全防御三个角度总结已有方法；最后讨论与本文关系最为直接的意图决策对齐研究，为后续问题定义和方法设计奠定基础。

2.1 智能体工具安全

大模型智能体的研究建立在大语言模型推理能力和外部工具执行能力结合的基础上。早期 **ReAct** 框架将推理与行动交替组织，使模型能够在生成中间推理的同时选择工具并利用环境反馈推进任务^[1]。随后，大模型智能体逐渐形成相对稳定的系统形态：核心语言模型负责理解用户目标和规划行动，记忆模块保存历史状态，工具模块连接文件系统、网页、数据库、命令行或外部服务，环境反馈又反过来影响后续决策。相关综述将这类系统概括为由规划、记忆、工具使用和环境交互共同构成的自治智能体系统^[2]。

与传统聊天模型相比，智能体安全的关键变化在于模型输出不再停留在文本层面。聊天模型的不安全输出通常表现为有害建议、错误事实或误导性内容；智能体则会把模型决策进一步转化为真实工具调用。一个看似普通的文件写入、脚本执行或 **API** 请求，一旦作用于错误对象或超出授权范围，就可能造成环境状态变化。因此，智能体安全需要同时关注文本、计划和行动三个层次，而不能只检查最终自然语言回复。

近期工作已经明确指出，文本安全和工具调用安全之间存在明显断裂。**GAP Benchmark** 将这种断裂形式化为文本层安全与工具调用层安全之间的偏差，并在多个受监管领域中观察到模型在文本中拒绝危险请求、却仍通过工具调用执行相关动作的情况^[9]。这类结果说明，面向智能体的安全评估不能只检查自然语言回复，还必须检查即将发生的工具行动。由此也引出了意图决策对齐问题：工具调用本身可能不含显式危险关键词，但它是否符合用户真实授权，仍需要在行动发生前进行判断。

围绕工具行动安全这一核心问题，后续研究大致形成了安全评测、输入侧防护、模型侧安全增强、运行时防御和意图决策对齐等路线。图 2.1 概括了本章相关工作之间的关系：评测工作刻画攻击面和度量方式，输入侧方法减少不可信信息进入上下文，模型侧方法提升模型自身的安全判断能力，运行时方法在工具执行

边界进行约束，意图决策对齐则进一步关注用户目标、模型决策与工具行动之间的授权关系。



图 2.1 大模型智能体安全相关工作的研究脉络

2.2 提示注入与安全评测

提示注入（Prompt Injection）是大模型应用中最具代表性的攻击形式之一。在传统文本交互中，攻击者通常直接诱导模型忽略系统指令或输出违规内容；在智能体场景中，攻击面进一步扩展到网页、邮件、文档、工具返回结果和本地工作区文件等外部数据源。间接提示注入（Indirect Prompt Injection）利用的正是这种外部数据进入模型上下文的过程：攻击指令并非来自用户本人，而是隐藏在智能体为了完成任务而读取的环境信息中，进而影响模型后续计划和工具调用。

AgentDojo 是这一方向中影响较大的评测环境。它构造了包含真实工具、任务状态和动态交互过程的智能体环境，用于评估提示注入攻击和防御方法在任务完成率与攻击成功率之间的权衡^[5]。InjecAgent 则面向工具集成智能体系统，系统评估间接提示注入在不同工具和任务中的攻击效果^[7]。这些工作使智能体攻击评测从单轮文本问答推进到多工具、多状态的真实任务过程，也说明攻击指令可以通过工具结果或环境文件进入模型决策链路。

随着智能体能力增强，安全基准也开始覆盖更复杂的攻击面。Agent Security Bench 将攻击和防御形式化到系统提示、用户提示、工具、记忆和知识库等多个组件中，用于评估智能体系统在不同攻击通道下的安全性^[8]。Agent-SafetyBench 进一步从更广义的智能体安全角度评估模型的鲁棒性和风险感知能力，指出单纯依

赖防御提示词难以充分解决智能体安全问题^[16]。SafeAgent 则通过自动风险模拟和合成数据生成来增强智能体安全，为构造大规模安全训练和评测数据提供了另一种思路^[17]。

近期也有工作开始反思智能体安全评测本身的可靠性。传统评测通常依赖一次贪心运行或少量采样轨迹，容易遗漏低概率但具有实际风险的长尾行为。Lin 等人提出以搜索而非单纯采样的方式测量智能体安全，强调应在一定概率预算内探索多条可能轨迹，而不是只报告单次运行结果^[18]。这一趋势与本文实验设计中的多轮复测和真实场景验证是一致的：智能体安全不是静态输入输出分类问题，而是需要在动态轨迹中观察攻击是否真实触发、干预是否真实生效。

2.3 输入侧防护

提示注入防御最直观的思路是在输入进入模型前进行处理，包括系统提示强化、来源标注、数据隔离、输入过滤和注入片段定位等方法。这类方法的优势是部署简单，通常不需要改动智能体主体结构；其局限在于，智能体运行过程中会不断产生新的工具结果和中间状态，单次输入过滤难以覆盖完整执行链路。

CaMeL 是输入可信边界和系统结构防护方向中的代表工作。它通过将控制流与数据流分离，使不可信外部数据不能直接影响由可信用户请求决定的控制路径^[6]。这一方法的价值不在于判断某个工具动作是否与用户任务对齐，而在于从系统结构上限制不可信数据对决策过程的影响。不过，在多组件智能体系统中，即使规划组件无法直接读取外部数据，它仍可能在计划中把选择工具或目标的权力交给后续处理不可信数据的组件。由此可见，不可信数据边界和行动授权边界需要同时被建模。

也有工作从工具结果解析和攻击定位角度减轻间接注入风险。Yu 等人提出通过工具结果解析过滤注入内容，使模型在接收工具返回值时获得更干净、更结构化的数据^[19]。PromptLocate 则关注注入发生后的定位问题，试图在被污染数据中找出注入指令和注入数据所在片段，为取证分析和数据恢复提供依据^[20]。这些方法都围绕不可信输入展开，适合降低外部内容污染模型上下文的概率。

面向实际部署的护栏框架也在快速发展。LlamaFirewall 将提示攻击检测、智能体对齐检查和不安全代码分析组合为面向智能体的开源护栏系统，体现了工业界和开源社区对智能体运行时安全的重视^[21]。ClawGuard 则在工具调用边界处执行用户确认的任务约束，将间接注入防御转化为工具边界上的确定性访问控制^[22]。这些工作说明，安全防护正在从依赖模型自我遵循提示，转向在模型行动边界上建立可执行约束。

2.4 模型侧安全增强

模型侧安全增强试图通过训练或后训练过程改善模型自身的安全判断能力。与输入侧过滤不同，这一路线不只在上下文外部增加规则，而是让模型在生成规划、判断风险或选择拒绝时具备更稳定的安全行为。**GuardReasoner** 通过构造带有推理步骤的安全数据集并进行推理监督微调和偏好优化，训练专门的安全护栏模型，使其在多个安全任务中具备更强的解释性和泛化能力^[23]。**IntentionReasoner** 则使用带有意图推理、安全标签和安全改写的训练数据训练防护模型，在边界查询中先分析用户意图，再进行多级安全分类和选择性改写^[24]。这类工作表明，安全模型并不只能依赖关键词分类，显式意图推理可以成为降低误拒和提高泛化能力的重要机制。

也有工作直接面向智能体或多步工具使用进行安全后训练。**MOSAIC** 将多步工具使用中的安全决策显式建模为“计划、检查、行动或拒绝”的循环，并通过轨迹偏好比较训练模型学习何时行动、何时拒绝^[25]。**MoralReason** 将大模型智能体的道德决策对齐表述为跨分布泛化问题，使用推理级强化学习训练模型在新场景中应用稳定的道德推理框架^[26]。**Intent-FT** 则通过让模型在回答前推断指令背后的真实意图，提升模型抵抗越狱攻击和识别隐藏恶意意图的能力^[27]。

模型增强方法说明，安全判断并不只能依赖外部规则或关键词过滤，也可以通过训练让模型学习更稳定的意图推理、风险识别和拒绝决策能力。这一路线对智能体安全具有重要意义，尤其是在需要处理边界模糊请求和隐蔽恶意意图时，经过安全训练的模型往往比静态规则具备更强的语义理解能力。

不过，模型侧增强通常要求训练数据、模型参数或部署模型保持可控。对于 **OpenClaw** 这类开放式智能体运行框架，用户可能接入不同闭源 API 或本地模型，防御系统难以假设核心模型一定经过同样的安全训练。因此，系统仍需要在核心模型之外保留可部署、可替换的安全检查层。模型侧增强在这里提供的关键启发是：意图推理本身可以被显式建模，安全防御也必须同时权衡风险识别能力、误报控制和判断成本。

2.5 运行时安全防护

输入侧防御和模型侧增强提供了重要基础，但它们并不能完全替代运行时治理。对于能够持续规划和调用工具的智能体而言，更直接的保护方式是在运行时（Runtime）观察其计划、推理轨迹和工具调用，并在高风险行动执行前进行拦截。

运行时防御可以采用策略推理、护栏智能体、程序分析或轨迹异常检测等形式。**TrustAgent** 使用 **Agent Constitution** 在规划前、规划中和规划后注入与检查安全

约束, 强调安全策略应贯穿计划生成过程^[28]。GuardAgent 采用额外的防护智能体理解安全规范, 并基于知识增强推理对主智能体行为进行检查^[13]。ShieldAgent 面向安全策略推理和验证, 在受保护智能体行动前生成并验证屏蔽计划^[11]。这些方法的共同点在于引入额外判断机制, 使核心智能体不再单独承担安全决策。

另一类工作将智能体轨迹看作可分析对象。AgentArmor 将智能体运行轨迹转换为包含控制流、数据流和程序依赖关系的中间表示, 并通过类型系统检查敏感数据流和策略违反^[10]。TraceAegis 从执行轨迹中抽象层次化行为单元, 并用行为约束检测异常行动^[14]。这类方法借鉴了传统软件系统安全的思想, 强调智能体虽然由自然语言驱动, 但其工具调用和数据流仍然可以被结构化建模。

运行时防御还可以直接面向工具调用步骤。ToolSafe 构建 TS-Bench 评测智能体逐步工具调用安全, 并提出 TS-Guard 与 TS-Flow, 在每一步行动前判断请求危害性和动作攻击相关性^[29]。Thought-Aligner 则从智能体内部思考过程出发, 对高风险思考进行动态修正, 再将修正后的思考反馈给智能体以影响后续行动^[30]。MAGE 进一步关注长周期威胁, 通过维护安全相关的影子记忆, 在长轨迹中保留可能被主智能体遗忘的风险上下文^[31]。这些工作体现出一个共同趋势: 智能体安全防护正在从静态输入过滤转向执行过程中的持续监测和动态干预。

这些方法为智能体运行时安全提供了重要基础, 但它们关注的重点并不完全相同。程序分析和轨迹异常检测擅长发现数据流泄露、执行顺序异常或策略违反; 工具调用护栏擅长在危险动作发生前给出判定; 思考修正和安全记忆则强调影响模型后续决策。若要处理动作表面普通但授权关系异常的情况, 防御系统还需要进一步理解用户意图、智能体决策和工具参数之间的语义关系。

2.6 意图决策对齐

任务对齐方向与本文关系最为直接。Task Shield 将间接提示注入防御重新表述为任务对齐问题, 要求每条指令和每次工具调用都应当贡献于用户指定目标, 而不是服务于外部数据中的攻击目标^[15]。这一视角将安全判断从“是否出现恶意字符串”推进到“当前行动是否有助于完成用户任务”, 为智能体工具调用安全提供了更符合任务语义的判定标准。

GAP Benchmark 进一步强调, 工具调用安全需要独立于文本安全进行度量^[9]。在智能体系统中, 模型的自然语言回复可能与工具调用行为不一致; 因此, 只检查回复内容无法保证行动层安全。本文方法沿着这一方向继续推进: 不只比较模型文本态度与工具调用之间是否存在差异, 还进一步比较用户真实意图、智能体当前决策和待调用工具参数三者之间是否一致。

意图理解研究也为本文提供了重要启发。**Intent Mismatch** 指出，多轮对话中的性能下降并不完全来自模型能力不足，而是用户意图与模型解释之间存在结构性错配；该工作通过中介模型将用户输入转化为更明确的任务指令，以缓解模糊意图在多轮交互中的累积影响^[32]。在智能体场景中，类似问题会进一步影响工具调用：用户的模糊目标、上下文中的伪授权和模型自发补全的中间目标，都可能被智能体误认为真实任务边界。因此，意图分析不仅是提升任务完成率的交互技术，也是智能体安全防御中的关键前置步骤。

意图决策对齐机制将上述问题落实到工具调用前的运行时检查中。它关注的不是某一条输入中是否含有注入语句，也不是工具命令表面是否危险，而是用户本轮真实意图、智能体当前决策和待执行工具调用之间是否保持一致。对于本地工具化智能体而言，这种比较关系直接对应行动授权边界：当工具调用仍服务于用户目标时，系统应尽量保持任务流畅；当目标漂移、授权来源混淆或操作对象重映射出现时，系统应在执行前介入。

表 2.1 大模型智能体安全相关工作的主要脉络

研究方向	代表工作	主要关注	对本文启发
安全评测	AgentDojo、ASB、GAP 等	攻击触发、工具安全、轨迹风险	实验需观察真实行动
输入与数据防护	CaMeL、PromptLocate 等	不可信数据隔离、解析与定位	外部内容不应直接成为授权来源
模型安全增强	GuardReasoner、MO-SAIC、Intent-FT 等	安全推理、拒绝决策、意图识别	可为专门安全裁判提供训练方向
运行时护栏	AgentArmor、ToolSafe、LlamaFirewall 等	工具边界、轨迹分析、动作拦截	防御应接近工具执行点
意图与任务对齐	Task Shield、Intent Mismatch 等	用户目标、模型解释与工具调用关系	显式比较意图、决策和动作

2.7 本章小结

本章从大模型智能体的工具调用安全出发，梳理了提示注入攻击与智能体安全评测、输入侧防御、不可信数据处理、模型增强、运行时轨迹防御以及任务和意图对齐相关工作。整体来看，已有研究已经从文本安全逐步推进到行动安全，从静态提示词防御推进到模型后训练和运行时工具边界治理，从单次攻击检测推进到长周期轨迹分析。

这些工作共同说明，大模型智能体安全的关键不在于某一个固定位置的过滤，而在于整个执行链路中用户目标、外部数据、模型决策和工具行动之间的关系。对于本文关注的 **OpenClaw** 场景，智能体可以直接读取本地文件、执行命令和修改工

作区内容, 安全防御必须在工具调用前判断当前行动是否仍然符合用户真实授权。

第 3 章 意图决策对齐问题定义与总体框架

本章内容将在第 3 章写作阶段补充。

第 4 章 基于意图决策对齐的防御机制设计与实现

本章内容将在第 4 章写作阶段补充。

第 5 章 实验设计与结果分析

本章内容将在第 5 章写作阶段补充。

第 6 章 总结与展望

本章内容将在第 6 章写作阶段补充。

参考文献

- [1] YAO S, ZHAO J, YU D, et al. ReAct: Synergizing reasoning and acting in language models [C/OL]//Proceedings of the International Conference on Learning Representations. 2023. <https://arxiv.org/abs/2210.03629>.
- [2] WANG L, MA C, FENG X, et al. A survey on large language model based autonomous agents [A/OL]. 2025. arXiv: 2308.11432. <https://arxiv.org/abs/2308.11432>.
- [3] OpenClaw Contributors. OpenClaw: Personal AI assistant[EB/OL]. 2026[2026-05-22]. <https://github.com/openclaw/openclaw>.
- [4] ZHANG Y, DENG X, WU J, et al. AgentWard: A lifecycle security architecture for autonomous AI agents[A/OL]. 2026. arXiv: 2604.24657. <https://arxiv.org/abs/2604.24657>.
- [5] DEBENEDETTI E, ZHANG J, BALUNOVIC M, et al. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents[A/OL]. 2024. arXiv: 2406.13352. <https://arxiv.org/abs/2406.13352>.
- [6] DEBENEDETTI E, SHUMAILOV I, FAN T, et al. Defeating prompt injections by design [A/OL]. 2025. arXiv: 2503.18813. <https://arxiv.org/abs/2503.18813>.
- [7] ZHAN Q, LIANG Z, YING Z, et al. InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents[C/OL]//Findings of the Association for Computational Linguistics: ACL 2024. 2024. <https://arxiv.org/abs/2403.02691>.
- [8] ZHANG H, HUANG J, MEI K, et al. Agent security bench (ASB): Formalizing and benchmarking attacks and defenses in LLM-based agents[C/OL]//Proceedings of the International Conference on Learning Representations. 2025. <https://arxiv.org/abs/2410.02644>.
- [9] CARTAGENA A, TEIXEIRA A. Mind the GAP: Text safety does not transfer to tool-call safety in LLM agents[A/OL]. 2026. arXiv: 2602.16943. <https://arxiv.org/abs/2602.16943>.
- [10] WANG P, LIU Y, LU Y, et al. AgentArmor: Enforcing program analysis on agent runtime trace to defend against prompt injection[A/OL]. 2025. arXiv: 2508.01249. <https://arxiv.org/abs/2508.01249>.
- [11] CHEN Z, KANG M, LI B. ShieldAgent: Shielding agents via verifiable safety policy reasoning [A/OL]. 2025. arXiv: 2503.22738. <https://arxiv.org/abs/2503.22738>.
- [12] LUO W, DAI S, LIU X, et al. AGrail: A lifelong agent guardrail with effective and adaptive safety detection[A/OL]. 2025. arXiv: 2502.11448. <https://arxiv.org/abs/2502.11448>.
- [13] XIANG Z, ZHENG L, LI Y, et al. GuardAgent: Safeguard LLM agents by a guard agent via knowledge-enabled reasoning[A/OL]. 2024. arXiv: 2406.09187. <https://arxiv.org/abs/2406.09187>.
- [14] LIU J, RUAN B, YANG X, et al. TraceAegis: Securing LLM-based agents via hierarchical and behavioral anomaly detection[A/OL]. 2025. arXiv: 2510.11203. <https://arxiv.org/abs/2510.11203>.

- [15] JIA F, WU T, QIN X, et al. The task shield: Enforcing task alignment to defend against indirect prompt injection in LLM agents[A/OL]. 2024. arXiv: 2412.16682. <https://arxiv.org/abs/2412.16682>.
- [16] ZHANG Z, CUI S, LU Y, et al. Agent-SafetyBench: Evaluating the safety of LLM agents [A/OL]. 2024. arXiv: 2412.14470. <https://arxiv.org/abs/2412.14470>.
- [17] ZHOU X, WANG W, LU L, et al. SafeAgent: Safeguarding LLM agents via an automated risk simulator[A/OL]. 2025. arXiv: 2505.17735. <https://arxiv.org/abs/2505.17735>.
- [18] LIN S, SURI A, OPREA A, et al. Toward a principled framework for agent safety measurement [A/OL]. 2026. arXiv: 2605.01644. <https://arxiv.org/abs/2605.01644>.
- [19] YU Q, CHENG X, LIU C. Defense against indirect prompt injection via tool result parsing [A/OL]. 2026. arXiv: 2601.04795. <https://arxiv.org/abs/2601.04795>.
- [20] JIA Y, LIU Y, SHAO Z, et al. PromptLocate: Localizing prompt injection attacks[A/OL]. 2025. arXiv: 2510.12252. <https://arxiv.org/abs/2510.12252>.
- [21] CHENNABASAPPA S, NIKOLAIDIS C, SONG D, et al. LlamaFirewall: An open source guardrail system for building secure AI agents[A/OL]. 2025. arXiv: 2505.03574. <https://arxiv.org/abs/2505.03574>.
- [22] ZHAO W, LI Z, ZHANG P, et al. ClawGuard: A runtime security framework for tool-augmented LLM agents against indirect prompt injection[A/OL]. 2026. arXiv: 2604.11790. <https://arxiv.org/abs/2604.11790>.
- [23] LIU Y, GAO H, ZHAI S, et al. GuardReasoner: Towards reasoning-based LLM safeguards [A/OL]. 2025. arXiv: 2501.18492. <https://arxiv.org/abs/2501.18492>.
- [24] SHEN Y, HUANG Z, GUO Z, et al. IntentionReasoner: Facilitating adaptive LLM safeguards through intent reasoning and selective query refinement[A/OL]. 2025. arXiv: 2508.20151. <https://arxiv.org/abs/2508.20151>.
- [25] AGARWAL A, SIYAN G, PANDYA Y, et al. Learning when to act or refuse: Guarding agentic reasoning models for safe multi-step tool use[A/OL]. 2026. arXiv: 2603.03205. <https://arxiv.org/abs/2603.03205>.
- [26] AN Z, DU W. MoralReason: Generalizable moral decision alignment for LLM agents using reasoning-level reinforcement learning[A/OL]. 2025. arXiv: 2511.12271. <https://arxiv.org/abs/2511.12271>.
- [27] YEO W J, SATAPATHY R, CAMBRIA E. Mitigating jailbreaks with intent-aware LLMs [A/OL]. 2025. arXiv: 2508.12072. <https://arxiv.org/abs/2508.12072>.
- [28] HUA W, YANG X, JIN M, et al. TrustAgent: Towards safe and trustworthy LLM-based agents [C/OL]//Proceedings of the Conference on Empirical Methods in Natural Language Processing. 2024. <https://arxiv.org/abs/2402.01586>.
- [29] MOU Y, XUE Z, LI L, et al. ToolSafe: Enhancing tool invocation safety of LLM-based agents via proactive step-level guardrail and feedback[A/OL]. 2026. arXiv: 2601.10156. <https://arxiv.org/abs/2601.10156>.
- [30] JIANG C, PAN X, YANG M. Think twice before you act: Enhancing agent behavioral safety with thought correction[A/OL]. 2025. arXiv: 2505.11063. <https://arxiv.org/abs/2505.11063>.

- [31] WANG Y, JIANG T, LIANG J, et al. MAGE: Safeguarding LLM agents against long-horizon threats via shadow memory[A/OL]. 2026. arXiv: 2605.03228. <https://arxiv.org/abs/2605.03228>.
- [32] LIU G, ZHU F, FENG R, et al. Intent mismatch causes LLMs to get lost in multi-turn conversation[A/OL]. 2026. arXiv: 2602.07338. <https://arxiv.org/abs/2602.07338>.

附录 A 前期基准测试补充结果

本附录后续将整理本文前期在 AgentDojo 与 CaMeL 等基准上的测试结果，用于说明本文研究问题的形成过程。该部分不作为正文实验的主要结论，只作为研究背景和实验探索的补充材料。

附录 B 真实场景样例与模拟轨迹示例

本附录后续将选取部分真实 OpenClaw 场景样例和模拟轨迹样例，展示 README 间接注入、目标别名重映射、模糊授权和长周期任务漂移等测试类型的构造方式。

附录 C 实验配置与运行说明

本附录后续将整理 OpenClaw、AgentWard、基线方法（baseline）脚本、模型配置和实验结果文件组织方式，以提高实验的可复现性。

致 谢

致谢部分将在论文主体内容基本定稿后统一补充。

声 明

本人郑重声明：所呈交的综合论文训练论文，是本人在导师指导下，独立进行研究工作所取得的成果。尽我所知，除文中已经注明引用的内容外，本论文的研究成果不包含任何他人享有著作权的内容。对本论文所涉及的研究工作做出贡献的其他个人和集体，均已在文中以明确方式标明。

签 名：_____ 日 期：_____